

线程池要想用的好，还得看这款开源框架！

你是否在项目中使⤵线程池遇到过以下问题？

- 创建线程池核心参数不好评估，随着业务流量的波动，极有可能出现生产故障。
- 不支持优雅关闭，当项目关闭时，大量正在运行的线程池任务被丢弃。
- 不支持运行时监控，使用过程中业务无响应，不知道是不是线程池引起。
- 三方框架 RocketMQ、Dubbo 等线程池无法动态修改参数，修改后只能重启应用。

在真实业务场景中，线程池可能遇到的问题比这里描述的还要多，稀奇古怪。

笔者所经历过的项目，因为业务对线程池参数没有合理配置，就触发过几起生产事故。大概在 21 年 6 月份左右，开始在网上搜索动态线程池的项目。

在开源平台找了挺多动态线程池项目，从功能性以及健壮性而言，个人感觉不满足企业级应用。

再加上当时看了美团动态线程的文章，就对这个技术方向比较感兴趣，所以决定自己来造一个轻量级的轮子。

我觉得写一个偏中间件的框架，还能帮助用户解决实际问题，是一件很酷的事情。

- GitHub: <https://github.com/opengoofy/hippo4j>
- Gitee: <https://gitee.com/opengoofy/hippo4j>

核心功能

通过对 JDK 线程池的增强，以及扩展三方框架底层线程池等功能，为业务系统提高线上运行保障能力。

Hippo4j 框架提供以下功能支持：

1. 客户端应用运行时实时变更指定线程池核心参数，变更生效支持集群和单实例两种方式。

2. 线程池运行时异常报警，比如：线程池活跃度、阻塞队列容量水位较高，触发了拒绝策略以及任务运行时间超长等。
3. 定时任务（默认 5 秒）采集线程池运行数据，可上报 Prometheus、InfluxDB 等数据库，搭配 Grafana 做大屏展示。
4. 运行过程中支持实时查看线程池当前运行状态以及线程池内线程的堆栈信息。
5. 支持 Tomcat、Undertow 和 Jetty 容器线程池运行时查看和动态变更线程池配置。
6. 支持 Dubbo、Hystrix、Kafka、RabbitMQ、RocketMQ 等客户端线程池运行时数据查看和动态变更线程池配置。

应用场景

1. 动态调参

Google 或者百度搜索线程池和生产事故关键字，几页都放不下，这也间接说明了线程池是个很考验使用者技术功底的技术点。

那有没有一些技巧或者技术来尽量规避线程池使用上的问题？比如：线程池的配置应该如何选择？



juejin.cn

<https://juejin.cn> > 后端 ▾

一次线上线程池任务事故 - 稀土掘金

2022年1月21日 — 查看业务情况和线程池配置，发现并行执行的任务数是小于线程池最大线程数的。以. ... 凌晨1点突发致命生产事故，人工多线程来破局！



jianshu.com

<https://www.jianshu.com> > ... ▾

线程池使用的10个坑 - 简书

2022年8月29日 — 但是线程池使用不当，就可能会引发生产事故。今天就和大家聊聊！ 1.线程池默认使用无界队列，任务过多导致OOM 2.线程创建过多，导致OOM



lianglianglee.com

<https://learn.lianglianglee.com> > 专栏 > 03 线程池： ... ▾

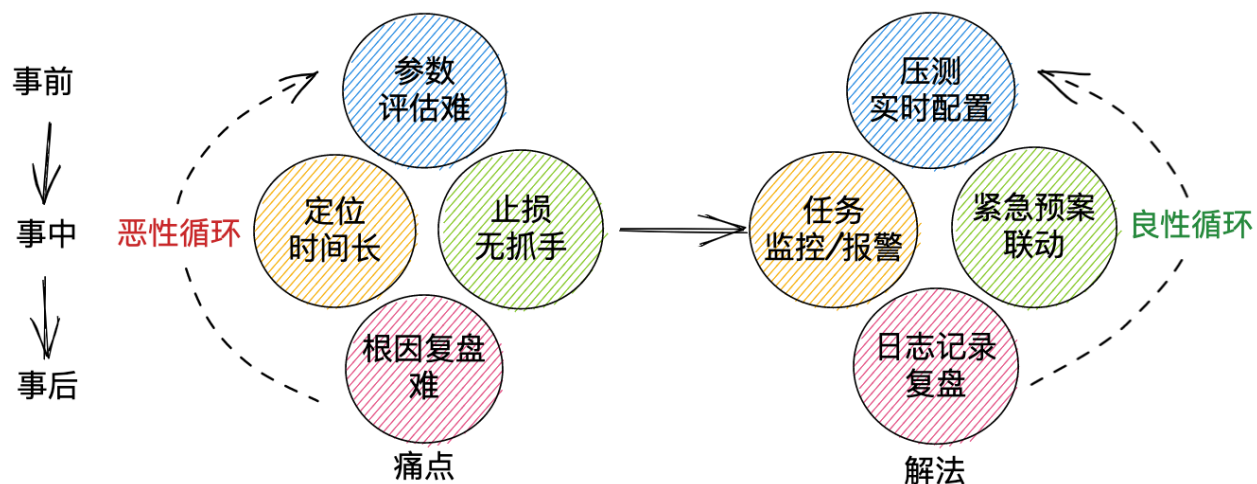
03 线程池：业务代码最常用也最容易犯错的组件.md

这一条规则的背后，是大量血淋淋的生产事故，最典型的就是newFixedThreadPool 和 newCachedThreadPool，可能因为资源耗尽导致OOM 问题。首先，我们来看一下 ...

我觉得大家对于线程池参数的纠结点主要有两个，无外乎设置的线程数多了或者少了。

- 如果预设的线程数或阻塞队列数量少了，当业务量上来，任务都在排队或者执行拒绝策略。
- 如果超量设置线程池的参数，无疑会造成资源浪费。

如果要修改运行中应用线程池参数，需要停止线上应用，调整成功后再发布，而这个过程异常的繁琐，如果能在运行中动态调整线程池的参数无疑会提高问题解决效率。



Hippo4j 提供了应用线程池运行时变更核心参数的功能。而且，如果应用是集群部署，可以选择修改线程池某一实例，或者修改集群全部实例，运行时生效，不需要再重启服务。

压测时可以使用 Hippo4j 动态调整线程池参数，判断线程池核心参数设置是否合理。对于开发测试来说，如果不满足可以随时调整。

* 核心线程

2

^

∨

* 最大线程

10

^

∨

* 队列类型

ResizableLinkedBlockingQueue (动态修改队列大小) ∨

* 队列容量

1024

^

∨

* 线程超时

超时

不超时

* 空闲回收

9999

^

∨

* 拒绝策略

SyncPutQueuePolicy ∨

全部修改

2. 告警策略

很多时候，线程池出故障的时候，系统已经发生了很严重的损失。有没有一种方式，在使用的线程池即将出现问题，但还算比较可控时，触发相关报警提示给用户，进而规避该问题？

Hippo4j 基于上述问题思考，集成了四种报警策略：

- 活跃度：假设阈值设置 80%，线程池最大线程数 10，当线程数达到 8 发起报警。
- 阻塞队列容量：假设阈值设置 80%，阻塞队列容量 100，当容量达到 80 发起报警。
- 触发拒绝策略：当线程池任务触发了拒绝策略时，发起拒绝策略报警。
- 任务运行超时：假设用户设置单个任务正常执行是 1000ms，实际执行超过该时间发起报警。

支持钉钉、企业微信和飞书软件通知，下图以线程池任务运行超时报警举例：

[警报] DEV - 动态线程池运行告警 (TIMEOUT)

线程池ID: **message-consume**

应用名称: **dynamic-threadpool-example**

应用实例: 192.168.1.5:8088_b7ab551b14204d37bb0f5cbea69f8974

核心线程数: 2

最大线程数: 10

当前线程数: 2

活跃线程数: 2

同存最大线程数: 2

线程池任务总量: 0

队列类型: ResizableCapacityLinkedBlockingQueue

队列容量: 1024

队列元素个数: 1

队列剩余个数: 1023

拒绝策略: SyncPutQueuePolicy

拒绝策略执行次数: **0**

任务执行时间: 1005 / ms

超时时间: 800 / ms

链路信息: 39948722194639841.251.16612352194691531

OWNER: **@马称**

提示: 5 分钟内此线程池不会重复告警 (可配置)

播报时间: 2022-08-23 20:16:12

3. 线程池监控

Hippo4j 线程池提供了两种监控方式: 线程池运行时数据采集监控以及客户端线程池运行实时状态查看。

1) 线程池核心参数监控。



2) 线程池实例运行时状态。

基础信息

线程池		环境标识	PROD	运行状态	Running
实例Host		实例标识	5f6616b32de84f42967ab01d1b730a89		

负载信息

当前负载	0%	峰值负载	66%	剩余内存	
内存占比					

线程信息

核心线程	4	当前线程	4	最大线程	6
活跃线程	0	同存最大线程	4		

队列信息

队列容量	4096	队列元素	0	队列剩余容量	4096
阻塞队列	ResizableCapacityLinkedBlockingQueue				

其它信息

任务总量	5147115	拒绝次数	0	最后更新时间	2023-03-09 19:40:13
拒绝策略	DiscardPolicy				

× 关闭

↻ 刷新

线程池运行时数据采集适合应用负责人巡查应用健康状态和排查问题时使用，实时状态适合排查多实例之间的运行数据状态。

4. 框架底层线程池

上面讲的动态线程池是业务中开发人员手动创建的线程池，比如下面这个：

```
@Bean
@DynamicThreadPool
public ThreadPoolExecutor messageConsumeDynamicExecutor() {
    String threadPoolId = "message-consume";
    return ThreadPoolBuilder.builder()
```

```
.threadFactory(threadPoolId)
.threadPoolId(threadPoolId)
.dynamicPool()
.build();
}
```

而框架线程池指的是某些三方中间件底层使用到的线程池，比如 Dubbo、RocketMQ 等框架，这些底层框架为了增强性能选择使用线程池进行扩展。

为什么要适配这些中间件框架的线程池？

相信这是很多小伙伴的疑问。以 Dubbo 举例，当服务高并发调用时，如果 Dubbo 底层线程池没有经过个性化配置，极有可能导致线程池打满，最终导致无法提供服务。

当遇到这种情况，可以使用 Hippo4j 对 Dubbo 线程池进行核心参数调整，避免生产故障时间持续。

再举个例子，当 RocketMQ 消息积压时，可能大部分公司的解决方案是添加客户端应用节点。而这种方式虽然可以解决问题，但是问题也很明显，太复杂且资源浪费。完全可以调整 RocketMQ SDK 底层线程池的线程数来达到快速消费的逻辑，有效解决 MQ 消息堆积问题。

目前 Hippo4j 已支持的三方中间件线程池列表：

- Apache Dubbo
- Alibaba Dubbo
- Apache Kafka
- Apache RocketMQ
- RabbitMQ
- SpringCloud Stream RocketMQ
- SpringCloud Hystrix
- Tomcat
- Jetty
- Undertow

上述中间件线程池都可以在 Hippo4j 页面上操作核心参数动态变更以及监控功能，如下所示：

运行报表		● RocketMQStream ×				
prescription 处方组		dynamic-threadpool-exam	input	搜索	重置	
序号	实例标识	Active	线程池标识	核心线程	最大线程	操作
1	192.168.1.5:8090_009716acb34846e495617bf85b49597d	DEV	input	1	2	编辑
2	192.168.1.5:8373_0842b25bd8cb4fc8beda1c66a0f5ab67	TEST	input	1	1	编辑
3	192.168.1.5:8374_f43c0dde46154ade8c9b594c2c0ee295	PROD	input	1	1	编辑

未来 Hippo4j 会支持更多三方框架线程池，如果你有好的想法也可以和我沟通，一起完善中间件框架适配。

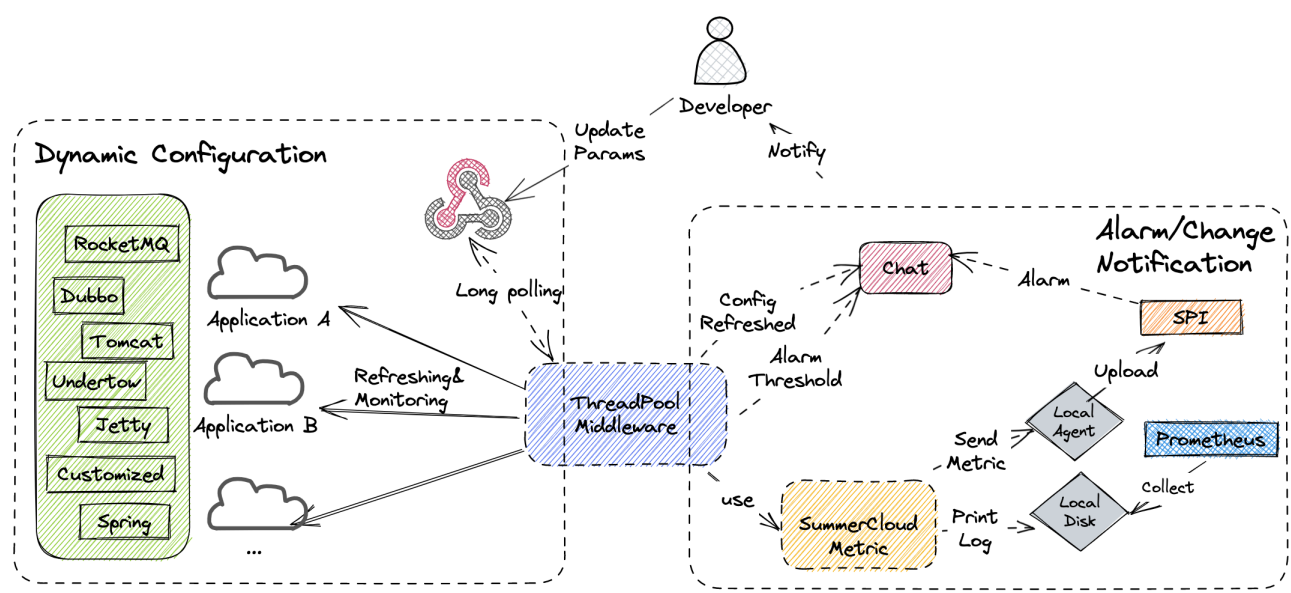
模块介绍



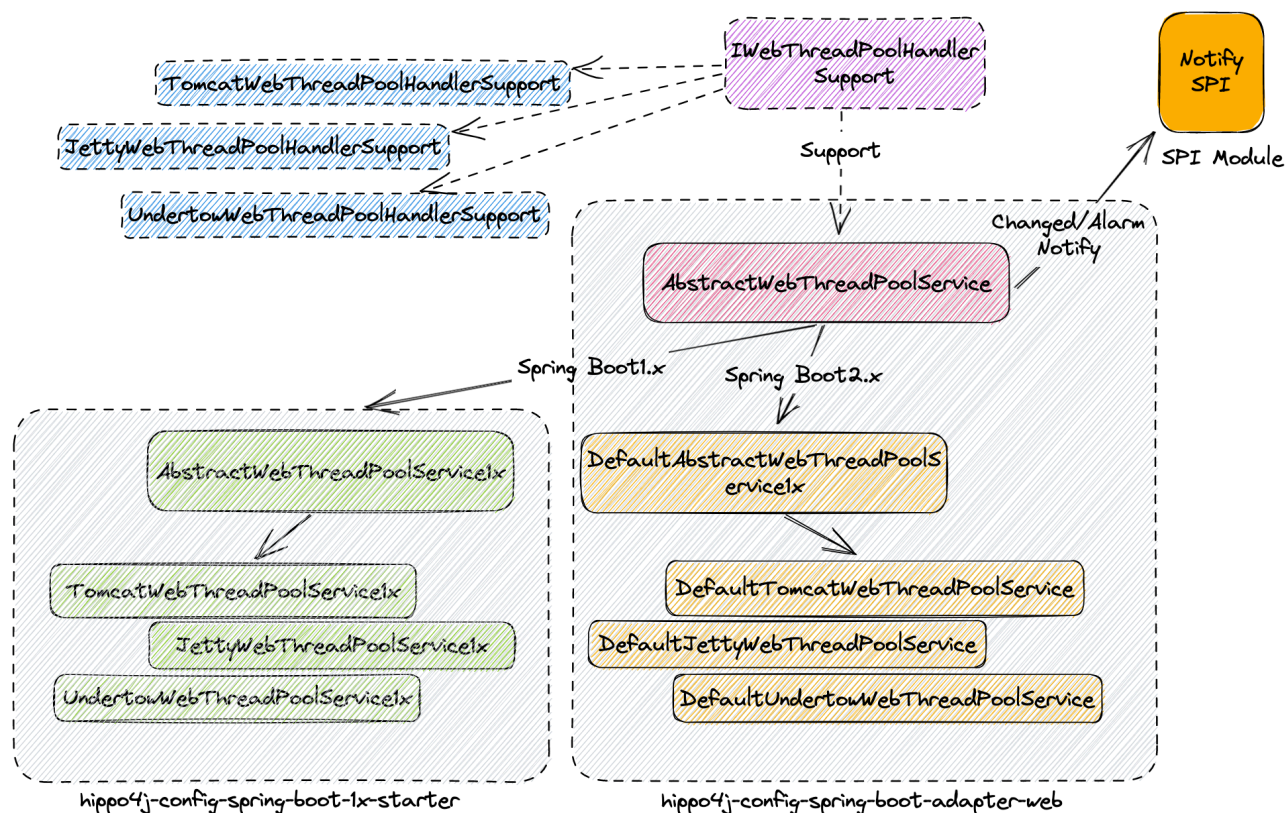
深入原理

如果一上来就下载 Hippo4j 的源码来看，很容易迷失进去。这里给大家画了几张图，帮助大家阅读源码时，能够抓紧主干分支，更快上手 Hippo4j 框架源码。

1. 配置中心模式变更原理



2. 适配 SpringBoot 1.5 & 2.x



3. 阅读源码优点

如果您公司没有使用 Hippo4j 场景的话，我也建议去阅读下项目的底层原理，主要有以下几个原因：

- 为了提高代码质量以及后续的扩展行为，运用多种设计模式实现高内聚、低耦合。
- 框架底层依赖 Spring 框架运行，并在源码中大量使用 Spring 相关功能。
- 运用 JUC 并发包下多种工具保障多线程运行安全，通过实际场景理解并发编程。
- 借鉴主流开源框架 Nacos、Eureka 实现轻量级配置中心和注册中心功能。
- 自定义 RPC 框架实现，封装 Netty 完成客户端/服务端网络通信优化。
- 通过 CheckStyle、Spotless 等插件规范代码编写，保障高质量代码行为和代码样式。

文末总结

Hippo4j 是 OpenGoofy 开源社区动态线程池框架，已有 32+ 公司生产实际使用经验，经历单节点连接数百应用考验。

至今已开源接近两年时间，期间发布 17 次 Release 版本，收获 Star 4.8k，10 位核心贡献者 Committer，101 位贡献者 Contributor。

- GitHub: <https://github.com/opengoofy/hippo4j>
- Gitee: <https://gitee.com/opengoofy/hippo4j>

更新: 2023-07-17 18:34:54

原文: <<https://www.yuque.com/magestack/12306/uoehltgxcewdckg8>>